BRAUDE
College of Engineering, karmiel

Software Engineering Department

Braude College

Capstone Project Phase B

Prevention Of Mold In Beehives

By:

Tamer Amer

Mbada Shehady

github link:

https://github.com/MbadaShehade/Prevention-of-mold-in-beehives.git

Project Code:
25-1-D-11

Advisor:

Dr. Naomi Unkelos Shpigel



CONTENTS

FIGURES **3**

1. Abstract **4**

2.Literature Review 5

3. PROJECT REVIEW AND PROCESS DESCRIPTION **5**

3.1. SYSTEM STRUCTURE 6

FIGURE 1 – Diagram to illustrate the work with the databases 6

3.1.1. General explanation of the system 7

4. THE ALGORITHM WE WERE BASED ON 15

5. TESTS **17**

5.1 Details of tests **18**

6. ACHIEVEMENTS 22

6.1. Challenges in the Process 23

7.1 User Guide 23

7.2 Programmer Guide 29

8. CONCLUSIONS / SUMMARY 37

8.1. Continue Development 37

8.2. SUS Questionnaire 38

9. REFERENCES **40**

FIGURES

FIGURE 1 – Diagram to show how the system functionality in general

FIGURE 2 – Introduction to our website

FIGURE 3 – The Problem

FIGURE 4 – Our Solution part 1

FIGURE 5 – Our Solution part 2

FIGURE 6 – Our Solution part 3

FIGURE 7 – More details about the system

FIGURE 8 – Beehive management

FIGURE 9 – Real time data from sensors

FIGURE 10 – PDF that user will get

FIGURE 11 – Real time graphs for humidity and temperature

FIGURE 12 – Historical Data graph

FIGURE 13 – Last air pump activations

FIGURE 14 – System FlowChart

1. Abstract

Imagine a beekeeper walking out to their hives after a long winter, only to find mold covering the combs and dead bees accumulated at the bottom. It happens often because beekeepers can’t see the changes happening inside the hive.

Mold inside beehives is not always immediately visible, but its effects can be devastating. It grows quickly under conditions of high humidity and poor airflow, contaminating stored pollen and honey, weakening the colony, and increasing stress levels among the bees[5]. This issue becomes even more critical during winter when natural ventilation is reduced. For beekeepers, especially those managing remote hives, monitoring such conditions is not only time-consuming but often impractical.

This project originated from the desire to create a more intelligent, automated, and user-friendly solution. Our goal was to give beekeepers a tool that would help them manage hive conditions remotely and effectively—without having to be physically present at all times. We designed and developed a complete IoT-based system that uses sensors to measure humidity and temperature inside the hive and automatically responds when certain environmental conditions conducive to mold appear.

The sensors, powered sustainably by a solar panel and backed by a battery, send data to the cloud via MQTT Explorer, where it is processed and visualized in real time on a web platform. The user interface, built with modern web technologies, offers a responsive and intuitive experience, allowing users to track hive data, view trends, receive alerts, and remotely activate the ventilation system

To make the system proactive, we implemented a rule-based mechanism that triggers an air pump when temperature and humidity exceed optimal thresholds—thus preventing mold before it starts. Beekeepers can also review historical data, compare multiple hives, and receive summarized reports that help them make informed decisions.

2.Literature Review

MQTT for IoT beehives:

MQTT is widely used in IoT because of its lightweight publish–subscribe model, making it more efficient and reliable than HTTP for remote, low-power beehive monitoring systems [1][2].

Temperature and humidity control in colonies:

Honey bee health depends strongly on stable hive conditions. Research shows optimal ranges of about 28–35 °C and 50–60%, with deviations increasing stress and risk of mold [3][4].

Mold risk and automated prevention:

Studies confirm that fungal contamination (e.g., *Aspergillus*) weakens colonies, contaminates honey/pollen, and stresses bees. Automated ventilation and monitoring help prevent these issues before they spread [5].

3. PROJECT REVIEW AND PROCESS DESCRIPTION

This project aims to develop a system that helps beekeepers prevent mold in their hives by monitoring environmental conditions in real time. Sensors placed inside the beehive continuously collect temperature and humidity data [3][4], which is transmitted via a solar-powered device using MQTT to the cloud [1][2]. The backend processes this data and presents it on a responsive web platform with clear graphs and alerts, while also triggering an air pump automatically when conditions exceed safe levels to improve airflow in the hive [6][7]. Together, this complete solution enables beekeepers to manage hive health remotely, reliably, and efficiently [5].

3.1. SYSTEM STRUCTURE

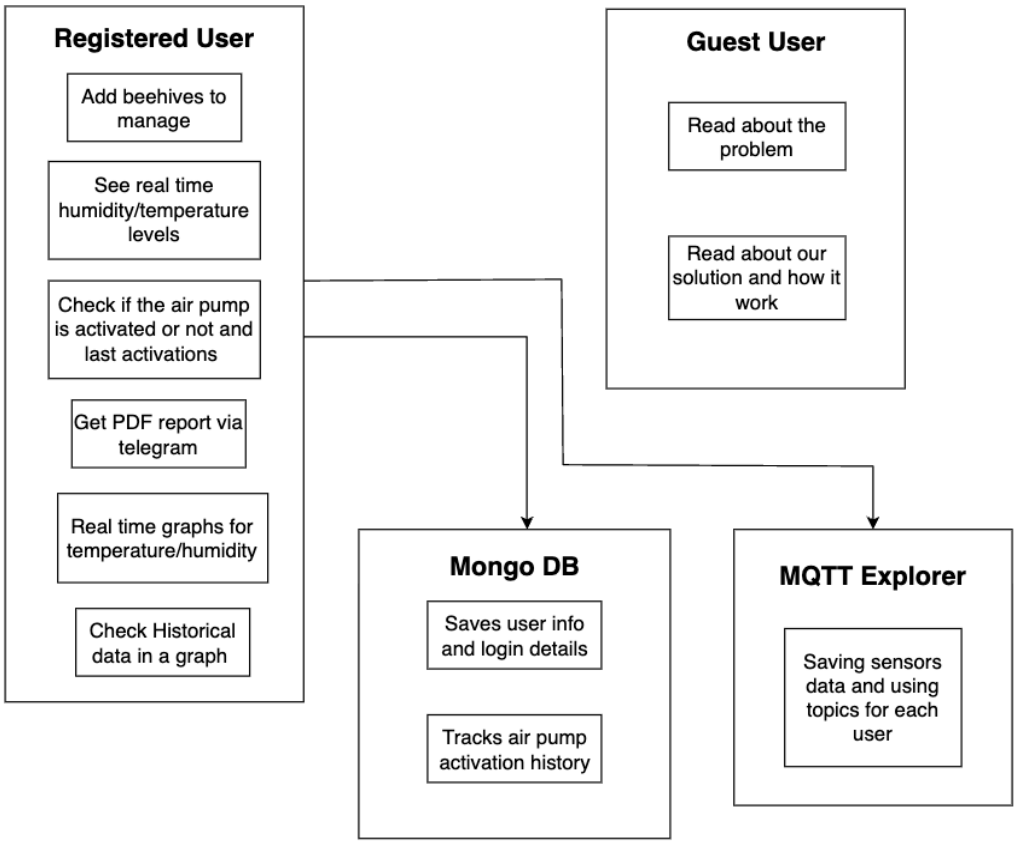


FIGURE 1 – Diagram to show how the system functionality in general

The use of MQTT over traditional HTTP-based APIs was driven by its lightweight, publish-subscribe model, ideal for low-power devices in remote locations. Unlike HTTP, MQTT enables continuous communication with minimal bandwidth consumption[1], which is critical in isolated beekeeping environments where cellular or Wi-Fi access may be limited or intermittent.

The system architecture is divided into four main parts:

1. **Beehive Hardware** – Temperature and humidity sensors collect data, while the air pump with a relay regulates airflow. Power is supplied by a solar panel and battery. The M5Stack device acts as the local controller.

2. **Communication** – Data is sent via 4G Portable Modem for wifi connection to the cloud. MQTT ensures reliable publish–subscribe transfer of sensor data.

3. **Backend & Storage** – MQTT Explorer manages the messages. MongoDB stores real-time and historical readings as well as pump activations.

4. **User Interface** – A web platform (React.js, Chart.js, Tailwind CSS) displays real-time data, history graphs, alerts, and reports. Visitors can only view general information, while registered users manage hives, receive PDF reports, and compare hive conditions.

3.1.1. General explanation of the system

You can log in to the system as a registered user or enter as a visitor.
Visitors can only see the introduction pages, while registered users can manage hives and view real-time data.

Website Introduction:

This screen introduces the system and explains its purpose. Visitors learn about the issue of mold in beehives and why monitoring humidity and temperature is important.



FIGURE 2 – Introduction to our website

The Problem:

This screen shows the risks of mold inside hives, including reduced airflow and high humidity that harm bee health. It explains why manual inspection is not enough.

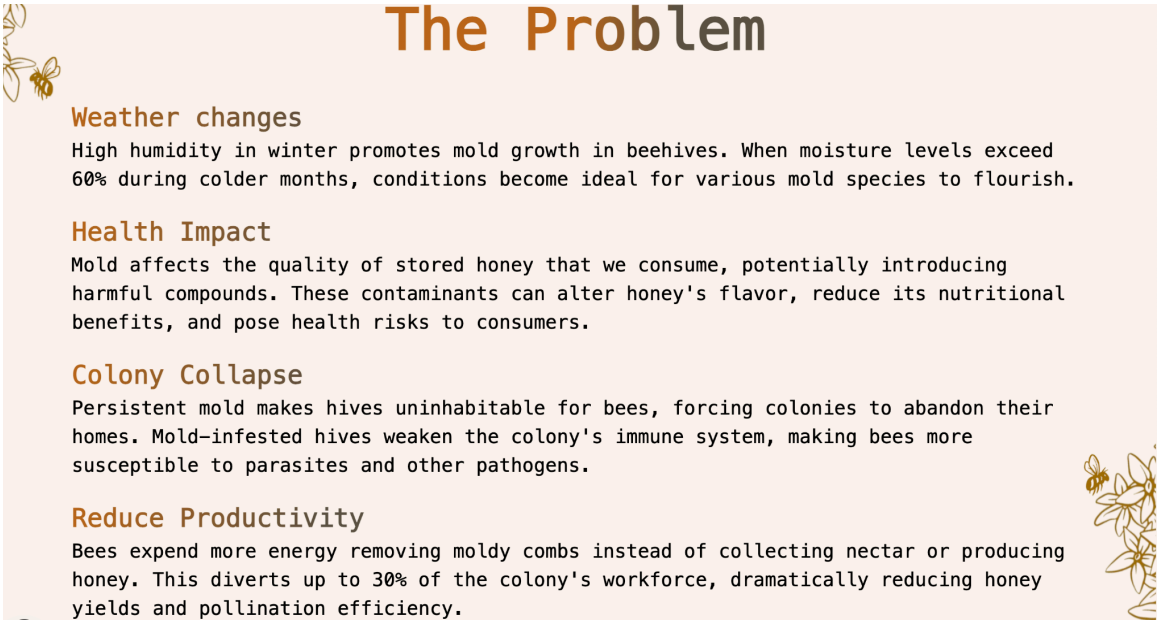


FIGURE 3 – The Problem

Our Solution (Part 1):

This screen presents the first part of our solution, showing how sensors inside the hive collect data and send it to the system.

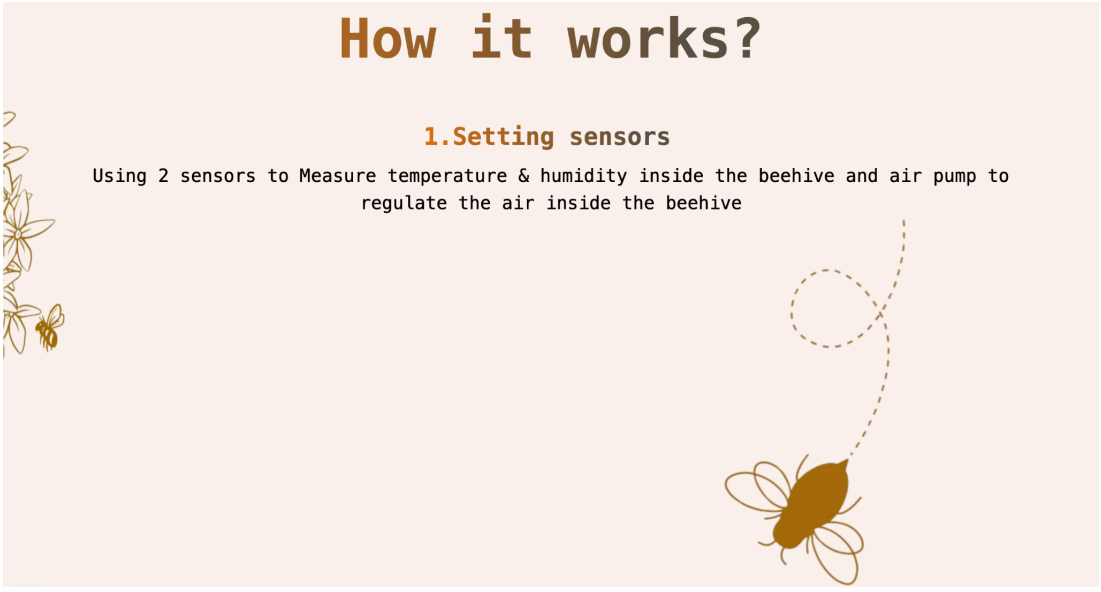


FIGURE 4 – Our Solution part 1

Our Solution (Part 2):

This screen explains how the system uses MQTT and cloud services to store and process hive data, even in remote areas.



FIGURE 5 – Our Solution part 2

Our Solution (Part 3):

This screen shows how beekeepers can access the data through a web platform with real-time monitoring and alerts.



FIGURE 6 – Our Solution part 3

More Details about the System:

This screen summarizes the architecture, including sensors, M5Stack, solar power, air pump, and data flow to the cloud.

The system begins with a solar panel that converts sunlight into electrical energy, ensuring that in low sunlight conditions, a battery can be used as an alternative power source.

An air pump will regulate airflow inside the beehive to maintain proper ventilation. Inside the beehive, two sensors will continuously measure humidity and temperature.

Data collected by the sensors will be transmitted using MQTT with an LoRa1 connection to an M5Stack device located in the beekeeper's house. This setup is necessary because the beehive's location lacks internet access. From the beekeeper's house, the data will be forwarded to the Mosquitto cloud service.

Once in the cloud, the data can be processed to generate graphs, charts, and additional insights for the beekeeper. The information will be accessible from anywhere.



FIGURE 7 – More details about the system

Beehive Management:

Registered users can create and manage multiple hives, each represented within a hexagonal layout for clear visualization. Users may add as many hives as they wish, with each hive monitored through sensors that track temperature, humidity, and air pump status. Hives are organized into groups of seven hexagons, and when a group is fully occupied, a new group with seven additional hexagons can be created. The dashboard provides a complete overview of all hive conditions while also offering an interactive comparison graph, enabling users to select specific hives and analyze their data side by side for deeper insights.

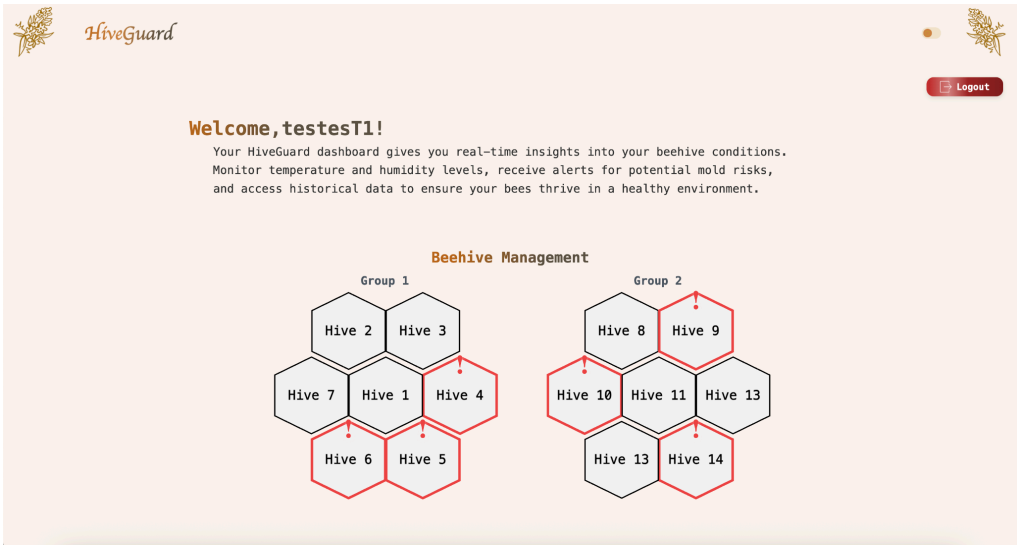




FIGURE 8 – Beehive management page (3 parts)

Now if the user click on a hive to see its condition we will go to new page as we see here:

Real-Time Data from Sensors:

This page allows the beekeeper to view the live conditions inside a specific hive. The screen displays the **current temperature and humidity** in large metric circles for quick visibility. Next to these values, the system also shows the **status of the air pump** (on/off) and the **duration of its last activation**. This is important because the beekeeper can immediately know if the hive is in a safe range (28–35°C and 50–60% humidity) or if intervention is required. The user doesn't need to refresh — the data updates

automatically, reflecting real-time sensor readings. By checking this screen, the beekeeper can confirm whether the automated system is actively regulating the environment and preventing mold.

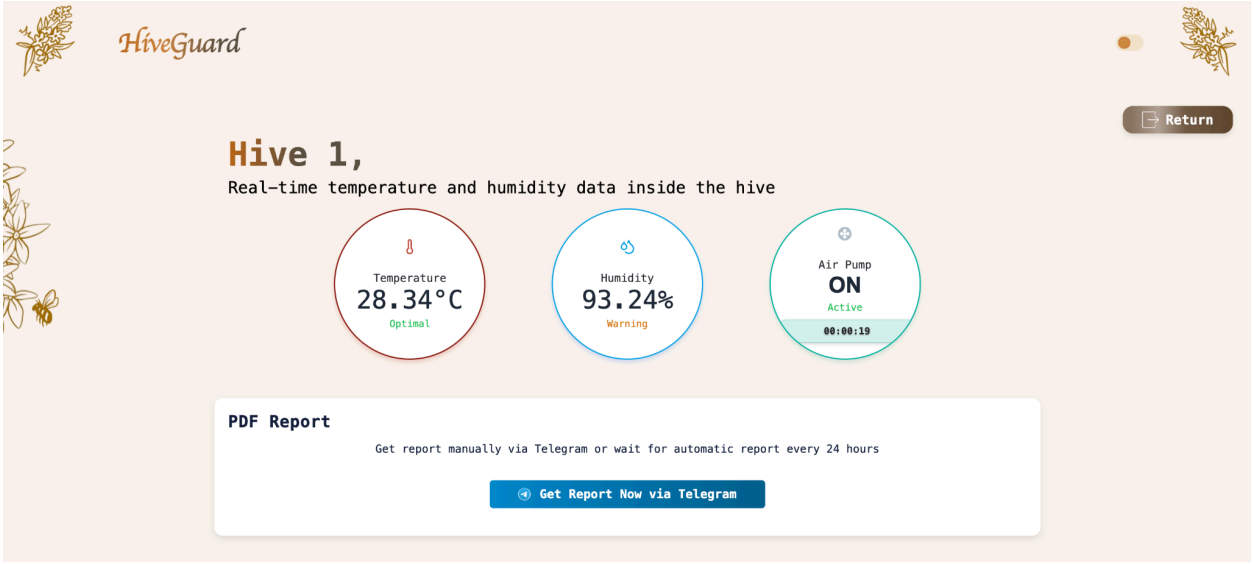


FIGURE 9 – Real time data from sensors

PDF Report:

Users can generate a PDF report of hive conditions, which is delivered directly to their Telegram account.

Beehive 1 Report

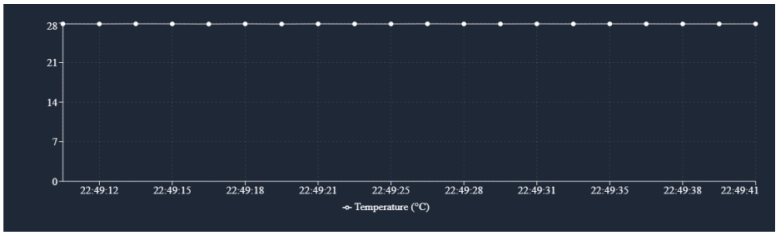
Generated for: testesT1

Generated on: 5/30/2025, 7:49:43 PM

Report type: Manual User Requested Report

Metric	Current Value	Status
Temperature	27.84°C	Optimal
Humidity	61.16%	Critical
Air Pump	OFF	Inactive

Real-Time Temperature Graph



Real-Time Humidity Graph

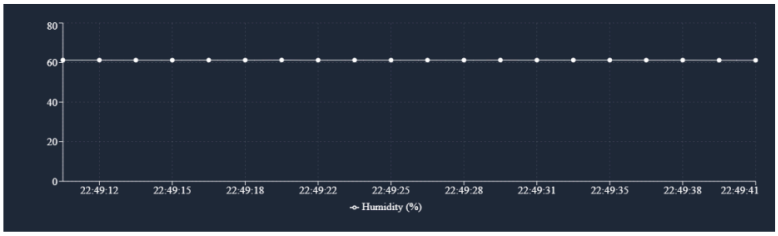


FIGURE 10 – PDF that user will get

Real-Time Graphs:

This screen shows dynamic graphs of temperature and humidity over time. The beekeeper can monitor changes minute by minute, helping them spot unusual trends or risks before they become problems.

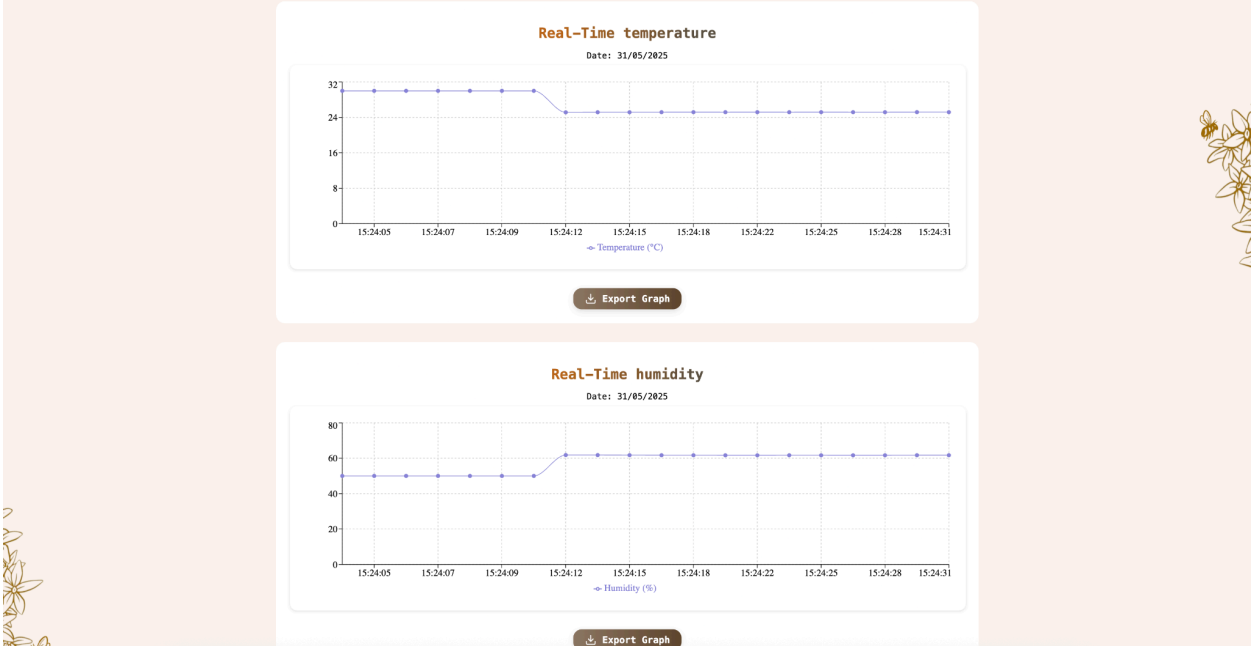


FIGURE 11 – Real time graphs for humidity and temperature

Historical Data Graph:

Here the user can view longer-term data trends. By looking at historical temperature and humidity levels, the beekeeper can compare seasonal patterns, evaluate interventions, and plan future actions.

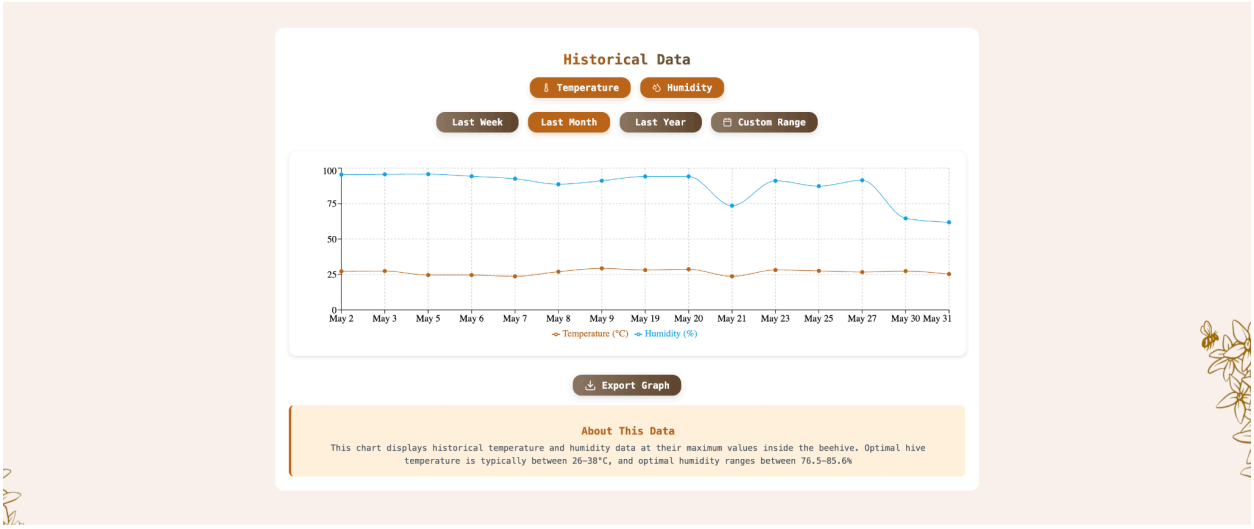


FIGURE 12 –Historical Data gra

Air Pump Activations:

This screen lists all past air pump activations, including date, time, and duration. It provides transparency, so beekeepers know how often the system is intervening. There is also an option to clear the history log if needed.

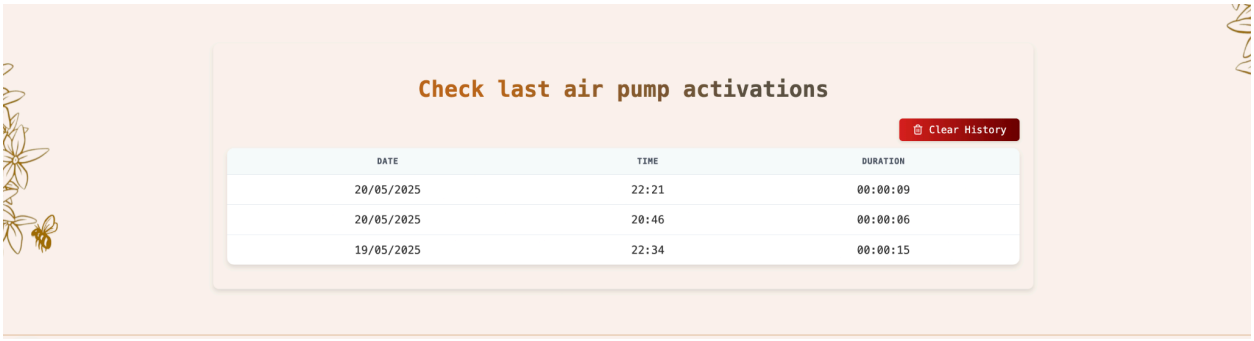


FIGURE 13 – Last air pump activations

4. THE ALGORITHM WE WERE BASED ON

Our system is based on a threshold-driven logic that reacts in real time to environmental changes inside the beehive. Using humidity and temperature sensors, the system continuously collects data from within the hive and evaluates it through logic blocks created in Flow.m5stack.com.

Based on scientific research, we established that the optimal temperature range inside a healthy beehive is between 28°C and 35°C, while the ideal humidity level ranges from 50% to 60%[3][4]. Whenever the sensor data goes outside these ranges, the system automatically activates an air pump to regulate airflow and reduce the risk of mold formation[6][7].

The sensors publish data to specific topics using the MQTT protocol[2]. These messages are accessed through MQTT Explorer, which acts as a middle layer in our architecture. Our system subscribes to these topics, allowing the website to receive the sensor data in real time and update the dashboard immediately.

Each air pump activation is recorded and stored. Users can see not only the real-time sensor data but also when the air pump was last activated and how long it operated. All this data is visualized through interactive graphs on the web interface.

This straightforward yet effective method ensures the beehive remains within healthy environmental conditions while giving beekeepers a powerful tool to monitor and manage their hives remotely.

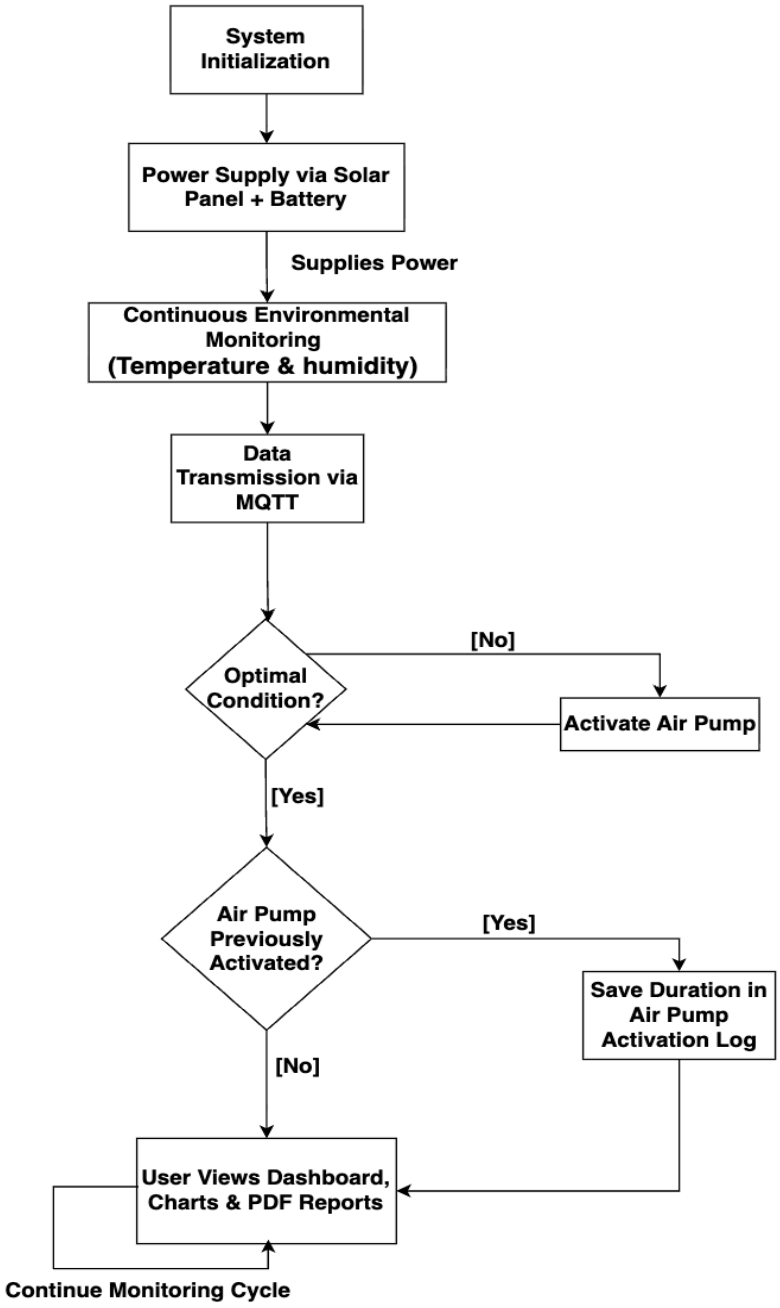


FIGURE 14 – System FlowChart

5. TESTS

As part of building our system, we needed to ensure that both the hardware and software components functioned correctly and reliably. Testing was performed at multiple stages, including real-time data flow, user interface actions, sensor accuracy, and system response to threshold violations. These tests were categorized into functional and non-functional tests.

Functional tests involved checking the registration and login processes for users, the ability to add and manage beehives, real-time display of temperature and humidity data, and the correct activation and logging of the air pump when thresholds were exceeded[8]. We also tested the generation of PDF reports via Telegram, as well as the real-time graphs and historical data views on the dashboard.

Non-functional tests included testing for stability and reliability. This involved simulating data transmission loss, sudden shutdown of the device, and network disconnections to ensure the system could recover gracefully[9]. We also tested how the system handled corrupted or incomplete sensor data, and whether alerts were sent appropriately when conditions were unsafe.

Particular attention was given to testing the integration between the M5 Stack Flow setup and the MQTT protocol, as any failure in communication could result in delayed or missing actions. We confirmed that messages were being published and received correctly, and that the front end reflected updates in real time.

Security testing involved checking that unauthorized users could not access registered user features and that database interactions (through MongoDB) did not expose sensitive user or hive data.

These tests helped us ensure that the system was robust and reliable for real-world usage, minimizing errors and giving beekeepers confidence in the technology managing their hives.

5.1 Details of tests

test #1	LOGIN			Result
#	**Action**	**Input**	**Expected Result**	Test Passed
1	Login with wrong user name	test5	Invalid username	
2	Login with wrong email	test5@gmail.com	User not found	
3	Login with wrong password	test	Invalid password	
4	Login with short user name	t	Username must be longer than 4 characters	
5	Invalid email format	test@gmail.co	Please enter a valid email address	
6	Normal login	Username: test5 Email: test@gmail.com Password: tewteW2	Success! Redirecting...	

test #2	Sign Up			Result
#	**Action**	**Input**	**Expected Result**	Test Passed
1	Sign Up with short user name	t	Username must be longer than 4 characters	
2	Using previous email	test@gmail.com	Username or email already exists. Please use a different username or email.	
3	Sign Up with short password	t	Password must be at least 7 characters, contain at least 1 number and 1 capital letter	
4	Sign Up with a username that was already used	testesT1	Username or email already exists. Please use a different username or email.	
5	Normal Sign Up	User name: Test1 Email: test@gmail.com Password: testesT1	Success! Redirecting...	

test #3	**Beehive Management**		Result
#	**Action**	**Expected Result**	Test Passed
1	Clicking on a hive that already was added but don't have sensors setup	No sensor data received. Please check your hive sensors.	
2	Clicking on a hive hexagon to add new hive	Hive added successfully	
3	Clicking on a hive hexagon that was added that have sensors setup	Navigating to hiveDetails page	
4	Adding more hives after a group is full	New group is added	

test #3	**HiveDetails Page**		Result
#	**Action**	**Expected Result**	Test Passed
1	Showing real time data	Viewing the data on the metrics circles and in graphs	
2	Valid Telegram Setup	Telegram chat ID saved successfully!	
3	Clicking on TelegramGet Report Now via Telegram	Hive report PDF sent to your Telegram successfully!	
4	Valid Telegram Setup	Telegram chat ID saved successfully!	
5	Showing historical data	Viewing the data on a graph	

6	Showing air pump activations	Viewing air pump activations in a table	
7	Trying to delete the log history of the air pump activations	No air pump activations yet.	
8	Telegram chat ID setup with letters	Telegram chat ID must be a number. Please try again.	

test #3	**RECOVERY**		Result
#	**Action**	**Expected Result**	Test Passed
1	Closing the website after receiving new data	Saving all relevant data	
2	Turn off computer immediately after registration	Saving all relevant data	
3	Clicking "Logout" after entering Beehive Management	Saving all relevant data	

6. ACHIEVEMENTS

In creating this project, we made a meaningful contribution to the world of beekeeping by offering a smart, automated solution that prevented mold and promoted hive health. Our achievements included:

1. Helping beekeepers maintain optimal hive conditions with minimal manual effort.

2. Reducing hive losses due to mold[7], especially during the winter season.

3. Increasing beekeeper confidence through real-time monitoring and reliable alerts.

We provided every registered user with a complete set of tools to manage their hives more effectively—by monitoring temperature and humidity, viewing historical data, and receiving reports.

From our point of view, success was measured by the beekeeper's ability to prevent mold without needing constant physical inspections, and by the adoption of our system as a trusted support tool in their regular beekeeping routine.

We evaluated the system with 15 beekeepers using the SUS questionnaire. The results showed that about 70% of users were satisfied with the system, 20% wanted additional features, and 10% encountered some challenges or required help with setup.

Overall, real user feedback demonstrated how the system helped beekeepers monitor and manage their hives more effectively. The responses showed that the system increased confidence, reduced the need for frequent physical inspections, and provided valuable insights for preventing mold. These results confirmed that our solution offered practical value and real support to the beekeeping community.

6.1. Challenges in the Process

During the implementation of our system, we encountered several technical and practical challenges that required careful attention and problem-solving.

One of the main challenges was integrating all hardware components—temperature sensor, humidity sensor, M5Stack, air pump, solar panel, and battery—into a unified and stable system that could operate outdoors and handle real-time data transmission.

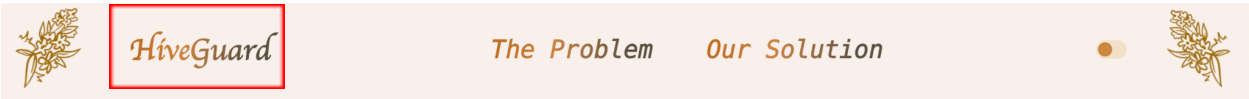
Another challenge involved ensuring that data would be transmitted smoothly via MQTT and received in real time without delays or loss. We needed to verify that our MQTT topics, subscriptions, and messages were handled correctly and consistently by both the sensors and the website.

A significant part of the project involved learning new platforms and tools such as **flow.m5stack.com**, which uses a visual block-based interface to define the behavior of the device. This required us to research, test, and debug complex logic blocks to achieve the correct threshold behavior.

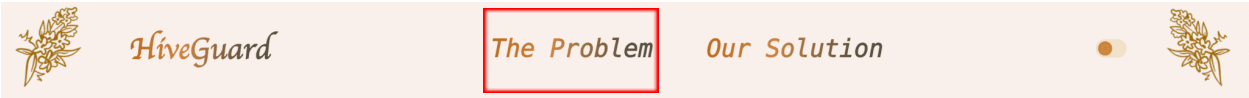
Despite these challenges, we were able to create a fully functional and stable system by iterating on the design, testing extensively, and making continuous improvements.

7.1 User Guide

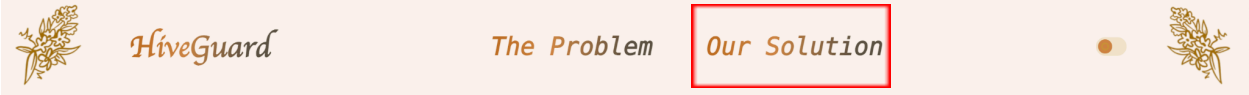
HiveGuard Button: clickable navigation to home when not logged in



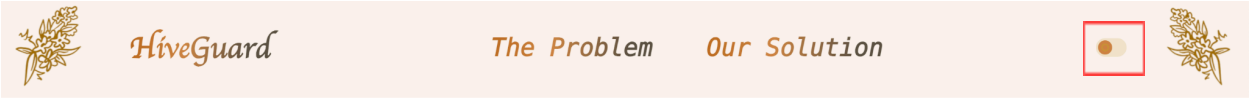
The Problem Button: navigate to “The problem” section below



Our Solution Button: navigate to “Our Solution” section below



Dark/light Mode Button: dark/light mode switching with animated sun/moon icon



Login Now Button: it navigates users to the /login page to access the system



Discover Our Solution Button: navigate down for “the solution” section



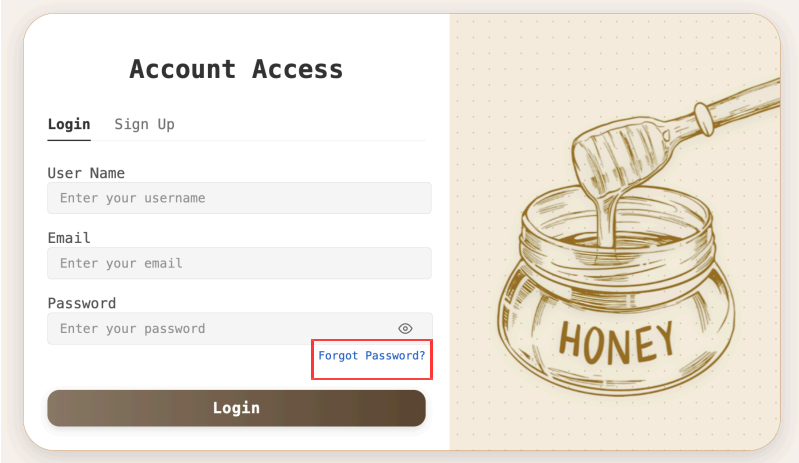
More Details Button: navigate down to see more details about the system



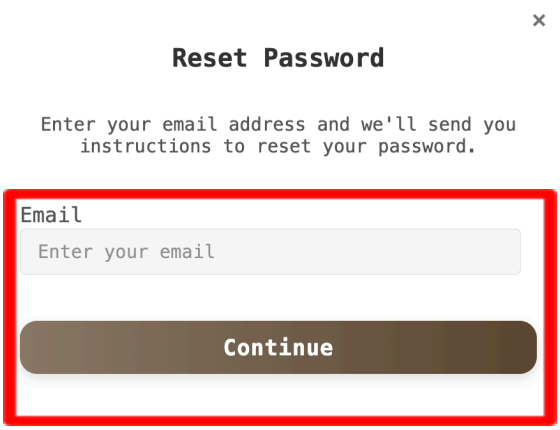
Swipe Up Button: navigate at the start of the page

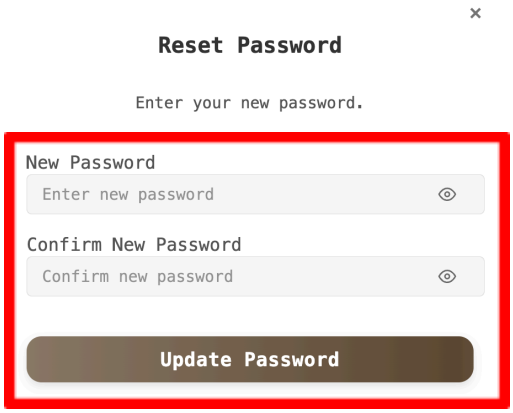


Login tab: For existing users to sign in by typing user name, email and password. As well as “forgot password” option

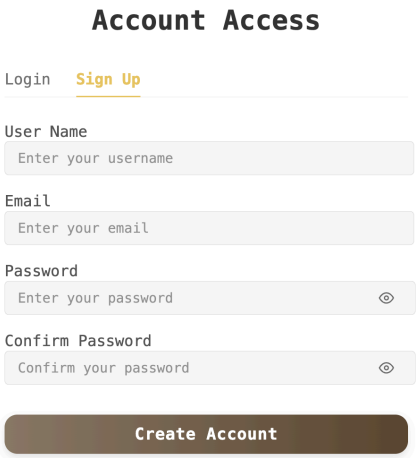


Forgot Password Button: in case if the user forgot the password





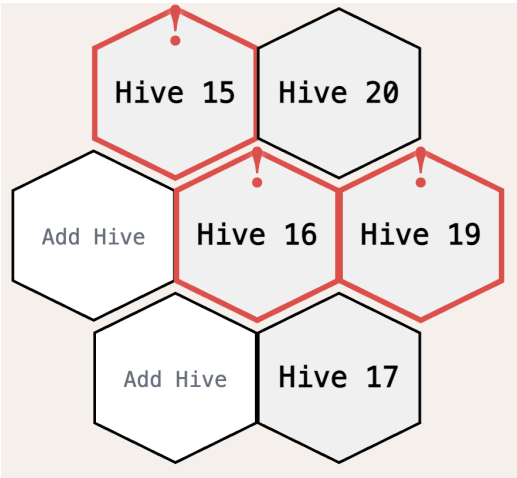
Sign Up Tab: For new users to create accounts by typing user name, email and password



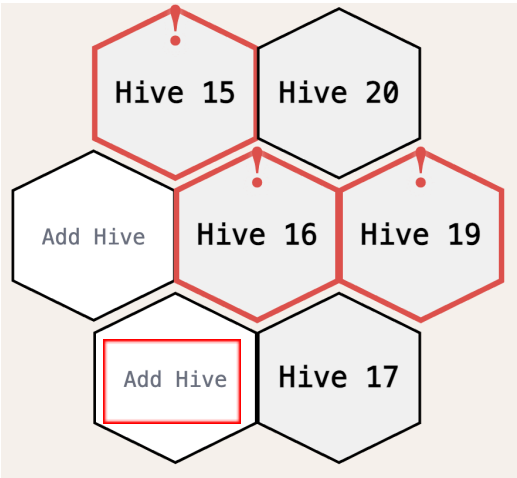
Logout Button: after logging in, users will have the option to log out, which will return them to the main page



Hexagon Button: When a user clicks on a hexagon, they are redirected to the specific hive information they want



Add Hive Button: When a user clicks on the 'Add Hive' hexagon, the system prompts them to either confirm adding a new hive or cancel the action

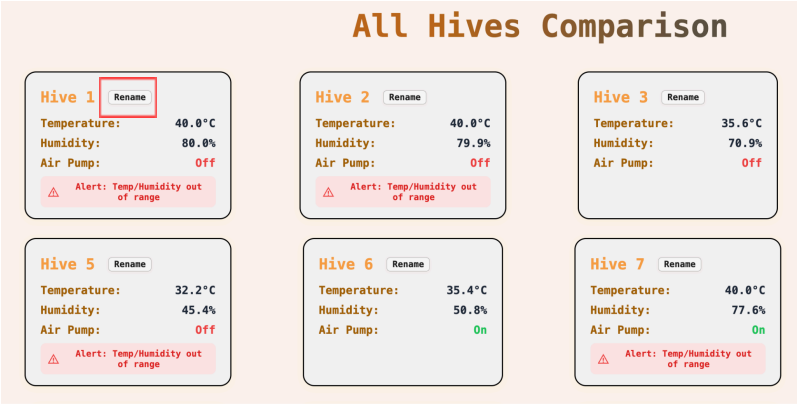


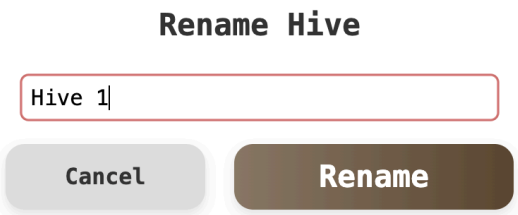


Add New Group Button: if there are 7 hexagons already in the most recent group an add new group Button appears so allow the user to add more hives to control

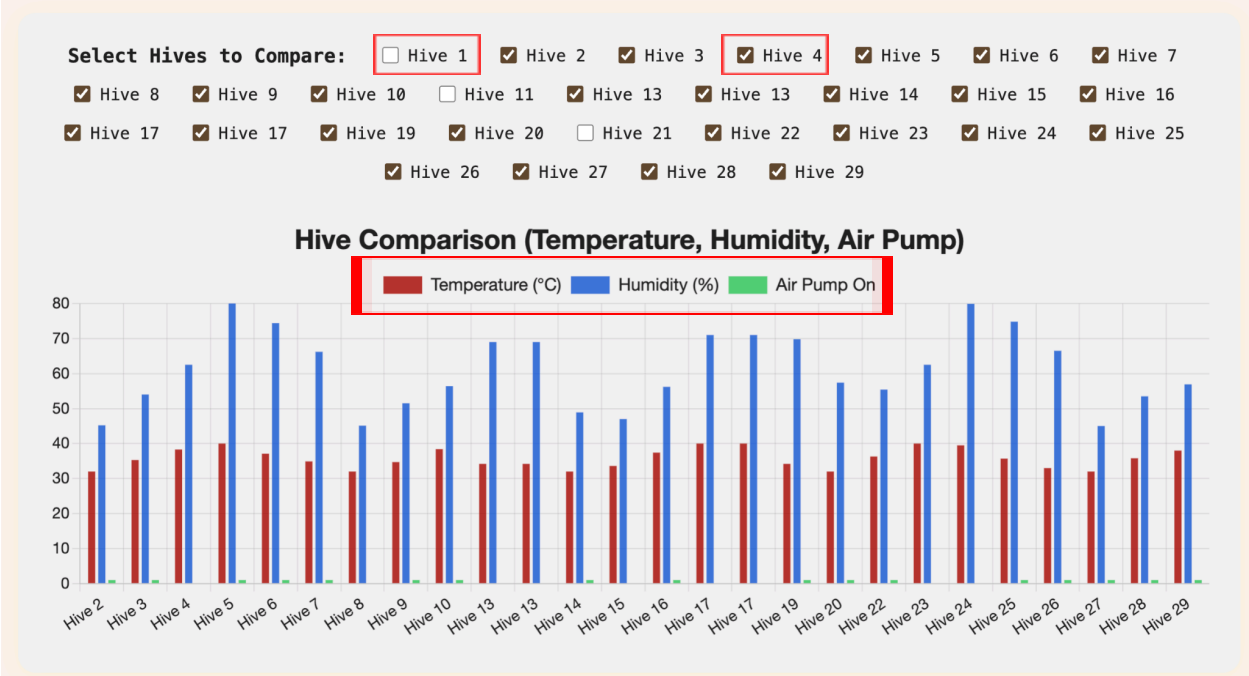


Rename Button: the rename button allow the user to change hives names

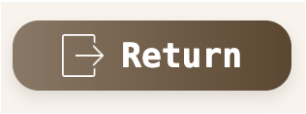




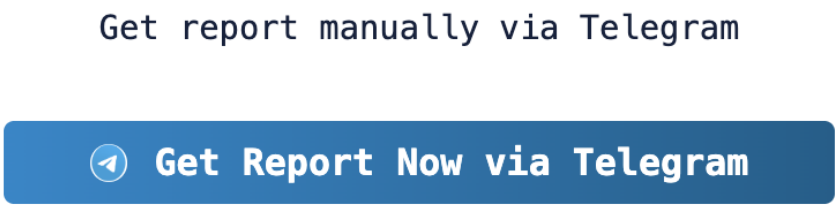
Hive Comparison Graph: Users can add or remove hives from the graph by clicking on hive names, as well as toggle the display of temperature, humidity, and air pump thresholds



Return Button: a return button in hive Details page after the user clicked on hive hexagon



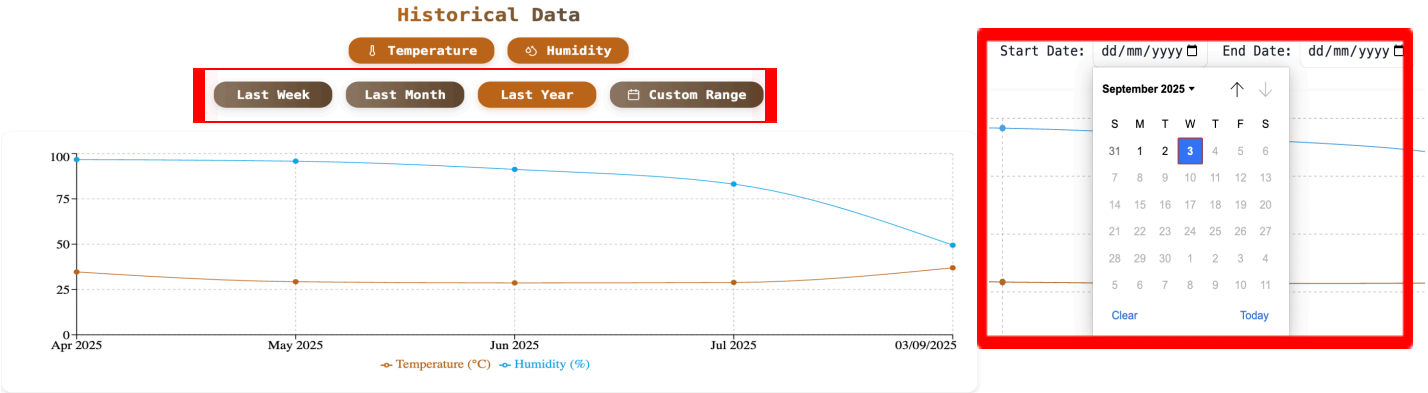
Telegram Report Button: Allow users to receive a PDF report via Telegram containing hive conditions and related graphs



Export Graph Button: allow the user to get an image of the graph in PNG/JPG formats



Historical Data graph: here the user can choose a duration time(last week, last month, last year or custom range) to see the last temperature/humidity levels in the hive.

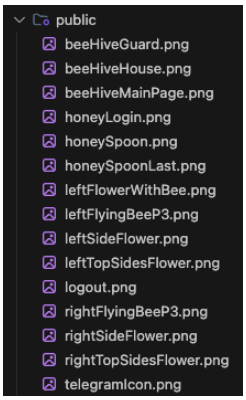


Clear History Button: clear all the last air pump activations in the log table



7.2 Programmer Guide

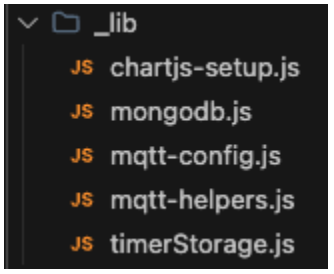
Key Files and Folders:
We used Next.js 15.2.1 to create our project. Next.js provides a powerful React framework with server-side rendering capabilities, which we used as the foundation for developing our IoT beehive monitoring application.



Public Folder:
The public folder contains static assets that are directly served to the client. In this project, it includes PNG images used for the website's branding, icons, and visual elements such as bee-themed decorations, hive illustrations, and UI icons for various functionalities like logout, air pump status, and navigation elements.

Src Folder:
The src folder contains the core source code for the application. It is organized into several subfolders and files that define the structure and functionality of the Next.js application. Below is a detailed description of each component and folder within the src directory.

1. _lib Folder:
The _lib folder is used for storing utility functions and configurations for external services such as MongoDB connection, MQTT helpers, chart.js setup, and timer storage utilities. It includes:



- **mongodb.js**:: Database connection configuration with connection pooling

```
32    if (dbInstance) {
33      return dbInstance;
34    }
35
36    const client = await clientPromise;
37    const dbName = process.env.MONGODB_DB || 'MoldInBeehives';
38    dbInstance = client.db(dbName);
```

- **mqtt-helpers.js**: MQTT service initialization and monitoring utilities

```
    const baseUrl = process.env.NEXT_PUBLIC_API_URL || '';
    const url = baseUrl ? `${baseUrl}/api/mqtt-monitor` : '/api/mqtt-monitor';

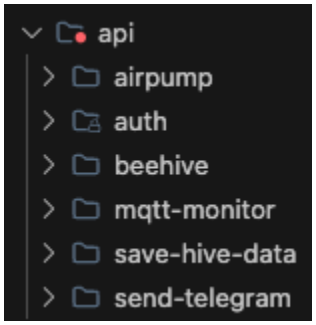
    // Fetch the monitoring service to ensure it's running
    const response = await fetch(url, {
      cache: 'no-store',
      method: 'GET'
    });

    const data = await response.json();
    console.log('[MQTT Helper] Monitor service status:', data.status);

    return data;
  } catch (error) {
    console.error('[MQTT Helper] Failed to initialize MQTT monitor:', error);
    return { status: 'error', message: error.message };
  }
}
```

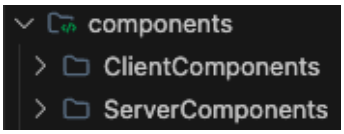
- **mqtt-config.js**: MQTT broker configuration settings
- **chartjs-setup.js**: Chart.js global configuration
- **timerStorage.js**: Air pump timer state persistence utilities

2. api Folder:



The api folder contains Next.js API routes that handle server-side operations including:
- **beehive**: operations for hive management (add, rename, fetch, changes)
- **auth**: Authentication and Telegram integration endpoints
- **mqtt-monitor**: MQTT monitoring service endpoints
- **save-hive-data**: Hive sensor data persistence
- **send-telegram**: Telegram notification service
- **airpump:** Air pump activation tracking

3. components Folder:



The components folder contains React components organized into ClientComponents and ServerComponents, defining the structure and behavior of different parts of the user interface.These components are always rendered on the screen, they are called by the layout.js component across all the pages in the application. They are not affected by the user interactions in the application.

For the **ClientComponents** folder:

Header.jsx:

The Header.jsx component is responsible for rendering the website's header, including the HiveGuard logo, navigation elements, and responsive design features. It utilizes React hooks and conditional rendering to provide a dynamic and interactive user interface based on authentication state.
The Header.jsx component provides a responsive and interactive navigation experience for the application. It features the HiveGuard branding, navigation components, and decorative flower elements.

This component ensures a consistent and user-friendly navigation experience across different devices and authentication states.

Component Structure

Props:

- `isLoggedIn`: Boolean to determine if user is authenticated
- `hiveDetails`: Boolean to indicate if viewing hive details page

State:

- No local state is used in this component

Logic:

- Conditional logo behavior based on authentication state
- Dynamic navigation component rendering
- Decorative flower elements positioning

The Header.jsx component also features Navigation and decorative flower image components.

Navigation.jsx:

It renders the navigation menu inside the header which displays different options based on authentication state:

- **Unauthenticated**: Shows login button and theme toggle
- **Authenticated**: Shows logout functionality and user-specific navigation
- **Hive Details**: Shows back navigation and specialized controls

Decorative Elements:

The header includes corner flower images that provide visual appeal and maintain the bee/nature theme of the application.

BeehiveManagement.jsx:

The BeehiveManagement component is the core component for managing multiple beehives in a hexagonal grid layout. It handles real-time data simulation, MQTT connections for live hives, hive addition/removal, and comprehensive data visualization.

```
const client = mqtt.connect('wss://test.mosquitto.org:8081', {
  clientId: `hiveguard_${Math.random().toString(16).substr(2, 8)}`,
  clean: true,
  reconnectPeriod: 0,
  connectTimeout: 30000,
  keepalive: 60,
  resubscribe: false,
  protocol: 'wss',
  qos: 1
});
```

Component Structure

Props:

- `email`: User's email for database operations
- `username`: Username for MQTT topic generation
- `password`: User password for navigation
- `hiveGroups`: Array of hive groups with their hives
- `setHiveGroups`: Function to update hive groups state
- `returnFromHive`: Boolean indicating return from hive details

State:

- `fakeHiveData`: Simulated sensor data for demo hives
- `fakeHiveHistory`: Historical data for chart rendering
- `selectedHiveIds`: Array of selected hives for comparison
- `showRenameModal`: Boolean for rename modal visibility
- `isLoading`: Loading state for hive operations

Key Functions:

- `addHive()`: Adds new hive to specified group position

```
const addHive = async (groupId, hexIndex) => {
  // Check if this position already has a hive in the current state
  const existingHive = hiveGroups.find(g => g.id === groupId)?.hives.find(h => h.position === hexIndex);
  if (existingHive) {
    console.log('This position already has a hive');
    return;
  }
  setPendingHiveAdd({ groupId, hexIndex });
  setShowAddHiveConfirm(true);
};
```

- `selectHive()`: Handles hive selection and data fetching

```
const selectHive = async (hive) => {
  // Track pressed hives uniquely by group and id
  setPressedHives(prev => {
    const exists = prev.some(h => h.id === hive.id && h.position === hive.position && h.name === hive.name);
    if (exists) return prev;
    return [...prev, hive];
  });
  console.log('Selecting hive with password:', password ? '[PRESENT]' : '[MISSING]');
  setSelectedHive(hive);
  setIsLoading(true);
```

- `Hexagon()`: Renders individual hexagonal hive representations

```
// Create a hexagon SVG with hive name
const Hexagon = ({ filled, label, onClick, alert }) => {
  const isDark = theme === 'dark';

  return (
    <div className="hexagon-container" onClick={onClick}>
      <svg viewBox="0 0 100 100" width="170" height="170">
        <polygon
          points="50 3, 95 25, 95 75, 50 97, 5 75, 5 25"
          stroke={alert ? '#ef4444' : (isDark ? '#e5e7eb' : 'black')}
          strokeWidth={alert ? '3' : '1.5'}
          fill={filled ? (isDark ? '#3a3a3a' : '#f0f0f0') : (isDark ? '#2d2d2d' : 'white')}
        />

        {alert && (
          <text
            x="50"
            y="23"
            textAnchor="middle"
            fontSize="32"
            fontWeight="bold"
            fill="#ef4444"
            style={{ filter: 'drop-shadow(0 1px 2px #fff8)' }}
          >
            !
          </text>
        )}
        {filled ? (
          <text
            x="50"
            y="55"
            textAnchor="middle"
            style={{ fontWeight: 510, fontFamily: 'FreeMono, monospace' }}
            fill={isDark ? "#f9fafb" : "black"}
          >
            {label}
          </text>
        ) : (
          <text
            x="50"
            y="55"
            textAnchor="middle"
            fill={isDark ? "#9ca3af" : "#6b7280"}
            style={{ fontSize: '12px', fontFamily: 'FreeMono, monospace' }}
          >
            Add Hive
          </text>
        )}
      </svg>
    </div>
  );
};
```

Hexagon Grid System:
The component uses a sophisticated hexagonal grid layout where each group can contain up to 7 hives (1 center + 6 surrounding). The positioning is calculated using trigonometric functions to create perfect hexagonal arrangements.

HiveDetails.jsx:
The HiveDetails component provides comprehensive monitoring and control for individual beehives. It includes real-time charts, historical data analysis, air pump controls, and Telegram reporting capabilities.

```
109     // Update real-time chart data (temperature)
110     setTemperatureData(prev => {
111       const newLabels = [...prev.labels, formatTime(new Date())].slice(-MAX_DATA_POINTS);
112       const newData = [...prev.datasets[0].data, newFakeData.temperature].slice(-MAX_DATA_POINTS);
113       return {
114         ...prev,
115         labels: newLabels,
116         datasets: [{ ...prev.datasets[0], data: newData }]
117       };
118     });
119     // Update real-time chart data (humidity)
120     setHumidityData(prev => {
```

Component Structure
Props:
- Retrieved from URL search parameters: `id`, `email`, `name`
State:
- `hiveData`: Current sensor readings (temperature, humidity, air pump status)
- `temperatureData`: Real-time temperature chart data
- `humidityData`: Real-time humidity chart data
- `historicalData`: Long-term historical data for analysis
- `airPumpActivations`: Array of air pump activation records
- `isAirPumpActive`: Boolean for air pump timer state
Key Functions:
- `startTimer()`: Begins air pump operation timing
- `stopTimer()`: Ends timing and saves activation record
- `sendReport()`: Generates and sends Telegram reports with charts
- `saveAirPumpActivation()`: Persists activation data to MongoDB

Real-time Data Simulation:
For demo purposes (non-primary hives), the component generates realistic sensor data using mathematical functions to simulate temperature and humidity fluctuations, including seasonal trends and random variations.

```
useEffect(() => {
  if (hiveId === '1') return;
  // More dynamic, realistic fake data simulation
  let temp = 30 + Math.random() * 5; // start 30-35
  let hum = 50 + Math.random() * 10; // start 50-60
  let airPumpOn = false;
  let lastPumpToggle = Date.now();
  let fakeActivations = [];
  let fakeHistory = [];
  let fakeHistorical = [];
  // Load from localStorage if available
  if (typeof window !== 'undefined') {
    const d = localStorage.getItem(`fakeHiveData_${hiveId}`);
    if (d) {
      const f = JSON.parse(d);
      temp = f.temperature || temp;
      hum = f.humidity || hum;
      airPumpOn = f.airPump === 'ON';
    }
    const h = localStorage.getItem(`fakeHiveHistory_${hiveId}`);
    if (h) fakeHistory = JSON.parse(h);
    const a = localStorage.getItem(`airPumpActivations_${hiveId}`);
    if (a) fakeActivations = JSON.parse(a);
    const hist = localStorage.getItem(`historicalData_hive_${hiveId}`);
    if (hist) fakeHistorical = JSON.parse(hist);
  }
```

LoginPage.jsx:

The LoginPage component handles user authentication with form validation, error handling, and secure password management using bcrypt hashing.

Component Structure

Props:

- No props passed to this component

State:

- `formData`: Object containing username, email, and password
- `errors`: Object containing validation error messages
- `isLoading`: Boolean for form submission state

Functions:

- `handleInputChange()`: Updates form data with validation
- `handleSubmit()`: Processes login with authentication
- `validateForm()`: Performs client-side form validation

Chart Components:

- **RealTimeTemperatureGraph.jsx**: Displays live temperature data with Chart.js
- **RealTimeHumidityGraph.jsx**: Shows real-time humidity measurements
- **HistoricalDataGraph.jsx**: Renders long-term data trends and analysis
- **HiveComparisonBarChart.jsx**: Compares multiple hives side-by-side

Modal Components:

- **TelegramModals.jsx**: Handles Telegram setup and error messaging
- **ClearHistoryModal.jsx**: Confirms historical data deletion

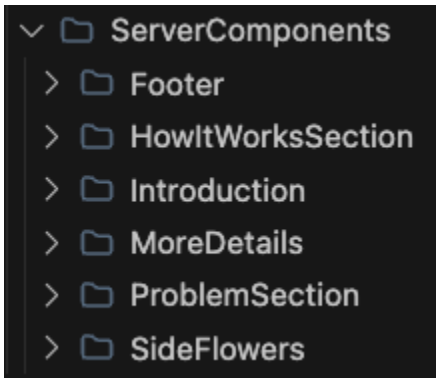
Theme and Animation Components:

- **ThemeToggle.jsx**: Switches between light and dark modes
- **FlowersRenderer.jsx**: Renders decorative elements for large screens

- **ScrollAnimations.jsx**: Handles page scroll effects and animations
- **SwipeUpButton.jsx**: Provides smooth scroll-to-top functionality

layout.js.jsx:
The layout.js component serves as the root layout for the Next.js application. It contains the main application structure, theme provider setup, font configurations, and global metadata. This component wraps all pages and provides consistent styling and functionality across the application.
page.jsx:
The page.js file serves as the main landing page component. It renders the introduction, problem section, how-it-works section, and other marketing components to showcase the HiveGuard system capabilities.
globals.css:
The globals.css file contains the global CSS styles applied throughout the application, including theme configurations, responsive design rules, and component-specific styling for both light and dark modes.

For the **ServerComponents** folder:
Footer.jsx:
The Footer component renders the footer section of the application. It includes contact information, social media links, and copyright notice.
The appearance of the footer changes based on the current theme (light or dark mode) of the application.



Component Structure
Props:
- No props are passed to this component
State:
- No local state is used in this component
Functions:
- Renders static footer content with contact information and branding

Introduction.jsx:
The Introduction component serves as the hero section of the landing page. It presents the main value proposition of HiveGuard with compelling visuals and a clear call-to-action for user engagement.
Functions:

- Renders hero content with title, description, and visual elements
- Displays main beehive illustration showcasing the monitoring system
- Integrates LoginButton component for user authentication
- Features decorative flower with bee image for visual appeal

ProblemSection.jsx:
The ProblemSection component outlines the key challenges that HiveGuard addresses. It presents four main problems related to mold in beehives with detailed explanations and supporting visuals.
Functions:

- Renders problem statements with structured content layout
- Explains weather-related humidity issues exceeding 60% in winter
- Details health impacts of mold on honey quality and consumer safety

- Describes colony collapse risks from mold-infested hives
- Outlines productivity reduction with 30% workforce diversion to mold removal
- Integrates DiscoverLink component for navigation to solution

HowItWorksSection.jsx:

The HowItWorksSection component explains the three-step process of how the HiveGuard system operates. It combines textual explanations with illustrative images to demonstrate the workflow from sensor placement to remote monitoring.

Functions:

- Renders step-by-step system workflow with visual aids
- Step 1: Setting sensors for temperature, humidity monitoring and air pump regulation
- Step 2: Data transmission via MQTT with LoRa1 connection to beekeeper's house
- Step 3: Remote monitoring capabilities allowing beekeepers to check conditions from anywhere
- Features flying bee illustrations for each step
- Includes beehive house illustration showing monitoring setup
- Integrates MoreDetailsLink component for additional information

MoreDetails.jsx:

The MoreDetails component provides in-depth technical information about the HiveGuard system architecture. It explains the power supply, data transmission, and cloud processing components with detailed technical specifications.

Functions:

- Renders detailed technical specifications and system architecture
- Explains solar panel power system with battery backup for low sunlight conditions
- Details air pump ventilation system and dual sensor setup for continuous monitoring

8. CONCLUSIONS / SUMMARY

Throughout the development of this project, we faced technical, logistical, and learning-related challenges, but the process provided us with valuable experience and meaningful results.

We had to explore new areas such as hardware-sensor integration, MQTT communication, real-time data visualization, and power-efficient outdoor systems. These were not part of our previous coursework, so we had to self-learn and troubleshoot along the way.

Working on this system gave us a deeper understanding of how environmental monitoring can be translated into real, working technology that directly supports agriculture and sustainability. It also emphasized the importance of testing, user experience, and simplicity in design.

In conclusion, we are proud of the final outcome. The system successfully monitors temperature and humidity inside the beehive and activates an air pump automatically when needed—reducing the risk of mold and improving hive health. The data is stored, visualized, and accessible through a clean and intuitive interface, empowering beekeepers with reliable insights.

8.1. Continue Development

While the current system meets its intended goals, we also identified several ideas for future development.

One area for improvement is to make the air pump activation more dynamic by allowing users to define custom threshold ranges directly from the website. This would give beekeepers more control over the environmental logic.

We also considered adding multi-hive support, where users could monitor several hives from a single dashboard and compare their environmental trends side by side.

Another potential enhancement is implementing notification features, such as SMS or Telegram alerts, to inform beekeepers when the air pump is activated or when extreme conditions are detected.

In the future, we may also explore offline data logging in case of internet failure, with automatic sync once the connection is restored.

These improvements could make the system even more adaptable, offering long-term value to a wider range of users.

8.2. SUS Questionnaire

To evaluate the usability of our system, we conducted a System Usability Scale (SUS) questionnaire with participating beekeepers. The SUS consists of 10 standard items, answered on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree):

1. I think that I would like to use this system frequently.
2. I found the system unnecessarily complex.
3. I thought the system was easy to use.
4. I think that I would need the support of a technical person to be able to use this system.
5. I found the various functions in this system were well integrated.
6. I thought there was too much inconsistency in this system.
7. I would imagine that most people would learn to use this system very quickly.
8. I found the system very cumbersome to use.
9. I felt very confident using the system.
10. I needed to learn a lot of things before I could get going with this system.

The SUS Score was: 82.343

Qualitative Analysis from users reviews:

Did anything about the interface confuse you during use?

"No, everything was clear, nice design."
This shows that the interface design met user expectations and felt natural to navigate. Since no confusion was reported, it suggests the layout and structure are intuitive even for non-technical users. This is especially important for beekeepers, who may not have advanced technical skills.

How did you feel about the system's responsiveness and loading speed?

"Responsive and stable on PC, a bit hard to see charts on phone."
The system works very smoothly on desktop, but the mobile version still needs some improvements, especially for viewing charts.

Would you feel comfortable using this system regularly to monitor hives remotely? Why or why not?

"Yes, because it saves time and lets me check hives without visiting them physically."
This response shows the **core value of the system is clear and relevant**: reducing the need for physical inspections. The fact that users feel comfortable integrating the system into their routine suggests high trust in both the data accuracy and usability. It reinforces that remote monitoring is not just a convenience but a **practical solution to real beekeeping challenges**, such as time management and hive accessibility.

Do you have any suggestions to improve the system (features, design, usability)?

"Multi-language support." , "More features could be nice, but the main purpose is well covered."
People are happy with the system as it is, but they'd like extra options like multi-language support and a few more features in the future.

What additional information or features would you like to see on this page?

"Location map." , "Camera to watch hive entrance activity."
This feedback shows a desire for **deeper environmental and behavioral monitoring**. A location map would help beekeepers keep track of multiple hives across different sites, improving management efficiency. A camera would provide direct observation of bee activity, complementing the sensor data.

9. REFERENCES

[1] EMQX. (2023). MQTT vs. HTTP: A protocol comparison. https://www.emqx.com/en/blog/mqtt-vs-http

[2] MQTT. (2024). MQTT protocol official documentation. https://mqtt.org/

[3] Abou-Shaara, H. F., Owayss, A. A., Ibrahim, Y. Y., & Basuny, N. K. (2017). A review of impacts of temperature and relative humidity on various activities of honey bees. *Insectes Sociaux, 64*(4), 455–463. https://doi.org/10.1007/s00040-017-0573-8

[4] BeekeepClub. (2023). How honeybees maintain temperature and humidity in a beehive. https://beekeepclub.com/how-honeybees-maintain-temperature-and-humidity-in-a-beehive/

[5] Becchimanzi, A., & Nicoletti, R. (2022). Aspergillus-bees: A dynamic symbiotic association. *Frontiers in Microbiology, 13*. https://doi.org/10.3389/fmicb.2022.968963

[6] Che Ali, M. A., Ilias, B., Abdul Rahim, N., Abdul Shukor, S. A., Adom, A. H., & Saad, M. A. (2021). A review on the stingless beehive conditions and parameters monitoring using IoT and machine learning. *Journal of Physics: Conference Series, 2107*(1), 012040. https://doi.org/10.1088/1742-6596/2107/1/012040

[7] Fernandes, K. E., Frost, E. A., Kratz, M., & Carter, D. A. (2024). Pollen products collected from honey bee hives experiencing minor stress have altered fungal communities and reduced antimicrobial properties. *FEMS Microbiology Ecology, 100*(7). https://doi.org/10.1093/femsec/fiae091

[8] BrowserStack. (2023). Guide to functional testing. https://www.browserstack.com/guide/functional-testing

[9] BrowserStack. (2023). Guide to non-functional testing. https://www.browserstack.com/guide/what-is-non-functional-testing